

МИНОБРНАУКИ РОССИИ

**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ «ЛЭТИ» ИМ. В.И.
УЛЬЯНОВА (ЛЕНИНА)**

КАФЕДРА ВТ

ОТЧЕТ

**по лабораторной работе №4
по дисциплине: «Операционные системы»
Тема: «Межпроцессное взаимодействие»**

Студент гр. 4314

Преподаватель

Пожидаев А. Н.

Тимофеев А. В.

Санкт-Петербург
2026

Цель работы: исследовать инструменты и механизмы взаимодействия процессов в Windows.

Задание 4.1 Реализация решения задачи о читателях-писателях.

Задание

1. **Постановка задачи:** Разработать консольные приложения «Читатель» и «Писатель» для моделирования классической задачи синхронизации при работе с общим ресурсом[cite: 3, 5].
2. **Организация буферной памяти:**
 - Взаимодействие процессов осуществляется через общую буферную память в виде проецируемого файла[cite: 8].
 - Размер одной страницы буфера должен соответствовать размеру физической страницы оперативной памяти[cite: 9].
 - Количество страниц в буфере определяется как сумма цифр номера студенческого билета (без учета первой цифры)[cite: 10].
 - Все страницы буферной памяти должны быть жестко заблокированы в ОЗУ с помощью функции VirtualLock[cite: 11].
3. **Алгоритм работы процессов:**
 - Длительность операций чтения и записи для каждого процесса выбирается случайным образом в интервале от 0,5 до 1,5 секунды[cite: 12, 15].
 - Синхронизация доступа к ресурсу должна быть реализована с помощью объектов Win32 API: семафоров или мьютексов[cite: 16, 17].
4. **Журналирование (логирование):**
 - Каждый экземпляр процесса обязан вести собственный журнальный файл[cite: 18].
 - В журнале фиксируются смены состояний: начало ожидания, выполнение операции (чтение/запись), переход к освобождению[cite: 18].
 - Для каждой записи фиксируется временная метка через TimeGetTime и номер используемой страницы буфера[cite: 18, 19].
5. **Проведение эксперимента:**
 - Необходимо обеспечить одновременную работу группы процессов, общее количество которых не меньше расчетного числа страниц буфера[cite: 20].

- На основании логов требуется построить графики смены состояний (минимум для 5 читателей и 5 писателей) и тепловую карту (heatmap) занятости страниц[cite: 21, 24, 25].

Описание реализации

Механизм синхронизации

Главным инструментом ограничения доступа к страницам является именованный семафор (префикс LocalPageWriteSem), который создается для каждой страницы отдельно.

- В данной задаче семафор используется в режиме двоичного замка (с максимальным значением 1).
- Для писателя этот семафор является абсолютным блокировщиком: пока писатель удерживает семафор, ни один другой процесс (ни читатель, ни другой писатель) не может получить доступ к этой странице.
- Для читателей этот же семафор служит индикатором присутствия писателя. Читатели захватывают его только коллективно, чтобы «застолбить» страницу и не дать писателю войти, пока идет процесс чтения.

Поскольку специфика задачи допускает одновременное чтение страницы несколькими процессами, возникает проблема: как понять, когда именно нужно заблокировать (или разблокировать) семафор для писателей? Для этого в разделяемой памяти заведен массив `readers_on_page`.

- Однако простое изменение переменной в памяти (`count++`) не является безопасным, так как два читателя могут сделать это одновременно, вызвав состояние гонки.
- Для решения этой проблемы используется именованный мьютекс (префикс LocalPageMutex). Читатель обязан захватить мьютекс перед любым изменением счетчика активных пользователей на странице. Это гарантирует, что только один процесс в конкретный момент времени решает, является ли он «первым вошедшим» (чтобы закрыть семафор от писателей) или «последним вышедшим» (чтобы его открыть).

Если рассматривать процесс синхронизации как поток событий, он выглядит следующим образом:

1. Запрос доступа: Процесс вызывает `WaitForSingleObject`. Если ресурс занят, ядро Windows переводит поток в состояние ожидания, полностью снимая его с выполнения процессором, что исключает «холостой» цикл.

2. Вход в критическую секцию: Как только объект синхронизации освобождается, один из ожидающих процессов (выбранный планировщиком ОС) получает управление. Для читателя это сопровождается захватом мьютекса, проверкой счетчика и, при необходимости, захватом семафора.
3. Безопасная работа: В состоянии активности процесс может быть уверен, что выбранная страница заблокирована согласно его правам (чтение или запись).
4. Освобождение: Процесс вызывает ReleaseMutex или ReleaseSemaphore. Это мгновенно переводит объект синхронизации в сигнальное состояние, пробуждая другие ожидающие процессы.

Читатели

Алгоритм «Читателя» более сложен, так как он поддерживает режим коллективного доступа к данным, блокируя только писателей. После стандартной процедуры инициализации и отображения файла в память, читатель вступает в цикл обработки данных.

В начале каждой итерации процесс выбирает случайную страницу и фиксирует в логе состояние ожидания (код 0). Для координации действий читатель использует два объекта синхронизации: мьютекс для защиты общего счетчика и семафор для блокировки записи. Сначала процесс захватывает именованный мьютекс, чтобы безопасно инкрементировать значение `readers_on_page` в разделяемой памяти. Если после инкремента выясняется, что данный читатель является первым на странице, он вызывает `WaitForSingleObject` для семафора записи, тем самым закрывая доступ писателям на все время, пока страница будет занята хотя бы одним читателем.

После освобождения мьютекса процесс переходит в состояние активности (код 1). Он считывает содержимое страницы, выводит его на экран и имитирует выполнение операции в течение заданного случайного интервала. По завершении чтения фиксируется состояние перехода к освобождению (код 2). Процесс снова захватывает мьютекс страницы и уменьшает счетчик активных читателей. Если текущий читатель оказывается последним (счетчик становится равен нулю), он вызывает `ReleaseSemaphore`, сигнализируя о том, что страница полностью свободна и доступна для писателей. В завершение итерации закрываются хендлы объектов синхронизации, и процесс готовится к новому циклу чтения.

Писатели

Процесс «Писатель» реализует стратегию строгого эксклюзивного доступа к ресурсу. Работа приложения начинается с инициализации системных параметров, где через функцию `GetSystemInfo` определяется размер физической страницы памяти. Далее выполняется проецирование файла `buffer_storage.bin` в адресное пространство процесса с обязательной фиксацией страниц в ОЗУ через `VirtualLock`.

Основной цикл работы писателя состоит из пятнадцати итераций. На каждом этапе процесс случайным образом выбирает целевую страницу буфера в диапазоне от 0 до 15. Перед началом модификации данных писатель переходит в состояние ожидания (код 0), запрашивая доступ к именованному семафору, уникальному для выбранной страницы. Использование функции `WaitForSingleObject` гарантирует, что запись начнется только тогда, когда на странице отсутствуют любые другие процессы — как читатели, так и другие писатели.

При получении доступа процесс переходит в состояние активности (код 1). В журнальный файл записывается метка времени `timeGetTime` и номер рабочей страницы. Писатель формирует информационную строку, содержащую его идентификатор и текущее время, после чего записывает её в буфер. Для имитации реальной нагрузки выполняется задержка от 0,5 до 1,5 секунд. Завершив операцию, процесс регистрирует состояние освобождения (код 2), отпускает семафор функцией `ReleaseSemaphore` и делает короткую паузу перед следующей итерацией, позволяя другим процессам перехватить ресурс.

Журналирование

Журналирование в данной работе организовано как децентрализованная система сбора данных, где каждый активный процесс самостоятельно фиксирует свою активность в отдельном файле формата CSV. Это позволяет детально восстановить хронологию событий при последующем анализе и построении графиков.

Для исключения конфликтов при записи каждый процесс создает уникальный файл, в названии которого фигурирует роль приложения («Reader» или «Writer») и его системный идентификатор (PID). Например, файл может называться `Writer_1234.csv`. Данные записываются в текстовом формате, где значения разделены запятыми, что упрощает их последующий импорт в Excel или Python для визуализации.

Каждая строка журнала соответствует одному событию и содержит три обязательных поля:

1. **Временная метка (Timestamp):** Получается с помощью функции Win32 API `timeGetTime`, которая возвращает количество миллисекунд, прошедших с момента старта операционной системы. Это обеспечивает высокую точность синхронизации событий между разными процессами.
2. **Код состояния (State):** Числовой идентификатор, описывающий текущий этап работы процесса:
 - 0 — **Ожидание:** процесс запросил доступ к странице и заблокирован планировщиком.
 - 1 — **Активность:** процесс успешно захватил объекты синхронизации и выполняет чтение или запись данных.
 - 2 — **Освобождение:** процесс завершил работу с данными и готовится отпустить семафор или мьютекс.

3. **Номер страницы:** Индекс страницы (от 0 до 15), с которой в данный момент взаимодействует процесс.

Процесс записи реализован через вспомогательную функцию LogCsv, которая принимает поток файла, код состояния и номер страницы. Важной особенностью реализации является момент фиксации событий:

1. Событие **«Ожидание»** регистрируется непосредственно **перед** вызовом блокирующей функции WaitForSingleObject.
2. Событие **«Активность»** записывается сразу **после** выхода из функции ожидания, что позволяет точно замерить время простоя процесса в очереди.
3. Событие **«Освобождение»** фиксируется **перед** реальным отпусканием объектов синхронизации (ReleaseSemaphore / ReleaseMutex), чтобы гарантировать, что в логах период активности не перекрывается с моментом, когда страницу уже занял другой поток.

Собранные данные служат фундаментом для аналитической части отчета. На их основе строятся сводные графики состояний, позволяющие визуально оценить переходы процессов между ожиданием и работой и тепловые карты (Heatmaps) занятости страниц, которые наглядно демонстрируют распределение нагрузки на буферную память и эффективность механизмов синхронизации в предотвращении коллизий.

Результаты экспериментов

В ходе работы было проведено два независимых эксперимента, различающихся интенсивностью нагрузки на систему. Для каждого эксперимента был сформирован полный пакет отчетной документации, включающий 20 графиков смены состояний (для каждого из 10 читателей и 10 писателей) и 2 тепловые карты занятости страниц.

Эксперимент №1: Базовая проверка (15 итераций)

Первый этап тестирования был направлен на подтверждение корректности логики работы алгоритмов в штатном режиме. При 15 итерациях на каждый процесс общая продолжительность работы составила около 30–40 секунд.

Цель: Проверка отсутствия ошибок при создании и открытии объектов синхронизации, а также контроль корректности записи логов.

Наблюдения: На графиках и хитмапах данного этапа фиксируется стабильная работа без длительных задержек. Ресурсы освобождаются достаточно быстро, чтобы новые процессы могли занимать страницы без формирования глубоких очередей.



Рис: 1. График пермены состояний читателя рід 12280



Рис: 2. График пермены состояний читателя рід 13588



Рис: 3. График пермены состояний читателя рід 15388



Рис: 4. График пермены состояний читателя rid 16884

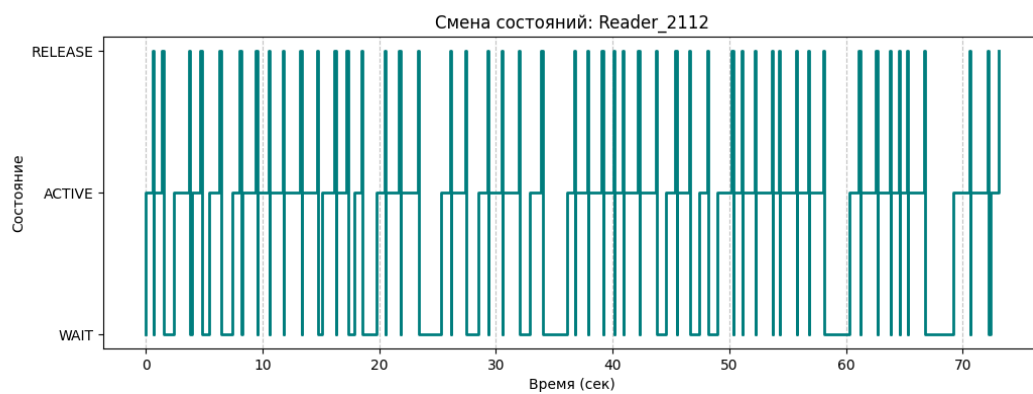


Рис: 5. График пермены состояний читателя rid 2112



Рис: 6. График пермены состояний читателя rid 2716



Рис: 7. График пермены состояний читателя рід 3052



Рис: 8. График пермены состояний читателя рід 3112



Рис: 9. График пермены состояний читателя рід 4960



Рис: 10. График пермены состояний читателя pid 7024



Рис: 11. График пермены состояний писателя pid 10120



Рис: 12. График пермены состояний писателя pid 13000



Рис: 13. График пермены состояний писателя pid 13104

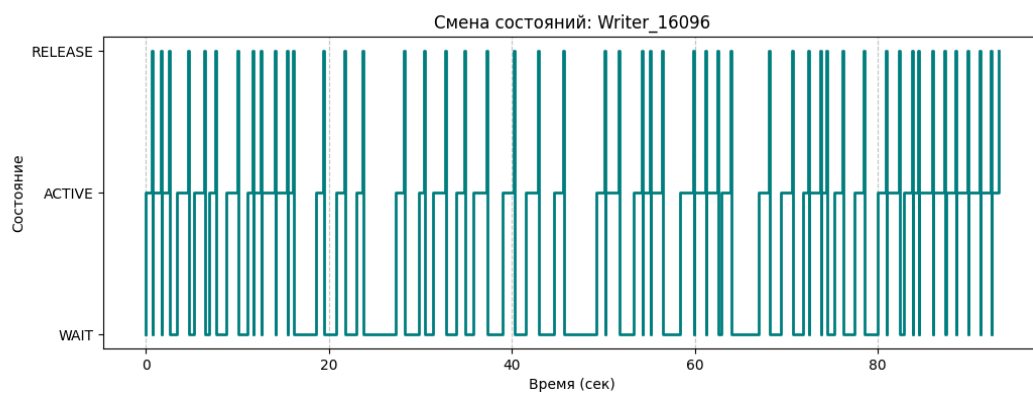


Рис: 14. График пермены состояний писателя pid 16096



Рис: 15. График пермены состояний писателя pid 4644



Рис: 16. График пермены состояний писателя pid 5076



Рис: 17. График пермены состояний писателя pid 5372



Рис: 18. График пермены состояний писателя pid 6092



Рис: 19. График пермены состояний писателя рід 6788



Рис: 20. График пермены состояний писателя рід 8860

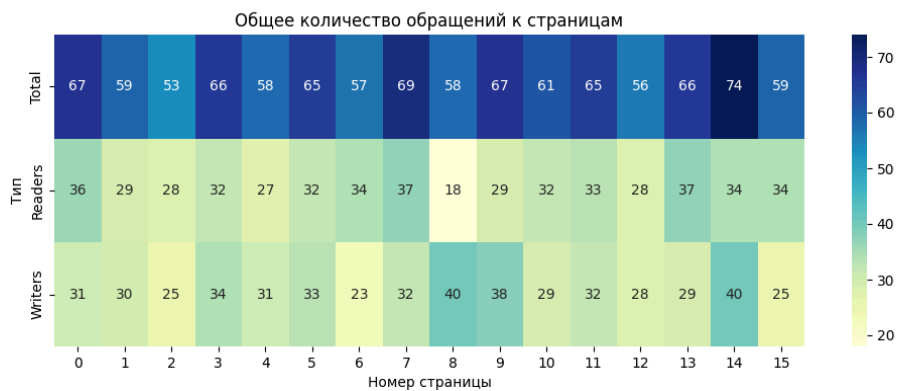


Рис: 21. heatmap обращений к старницам за всё время эксперимента 1

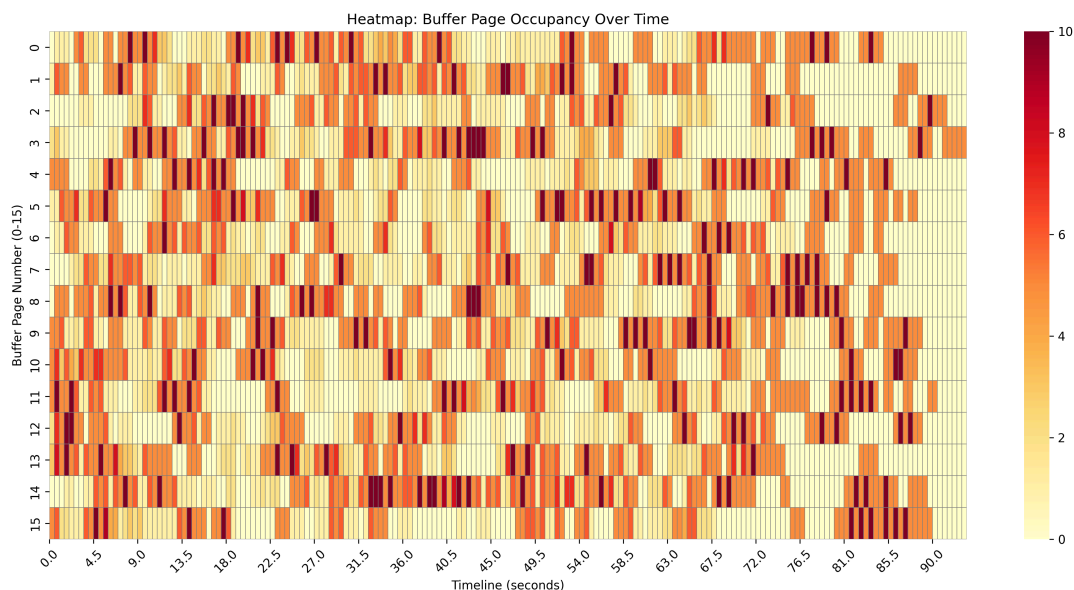


Рис: 22. heatmap обращений к страницам во времени в ходе эксперимента 1

Эксперимент №2: Расширенное тестирование (50 итераций)

Второй этап проводился при тех же 20 активных процессах, но с увеличением числа итераций до 50 для каждого. Это позволило перевести систему в режим длительного функционирования.

Цель: Изучение поведения механизмов синхронизации при накоплении запросов и проверка устойчивости проецируемого файла к длительным блокировкам.

Наблюдения: Увеличение объема данных позволило более наглядно визуализировать работу планировщика ОС. На графиках состояний отчетливо видны участвовавшие периоды ожидания (код 0), что характерно для условий высокой конкуренции за популярные страницы буфера. Тепловые карты за этот период демонстрируют более плотное заполнение временной шкалы, подтверждая надежность работы исключаящих семафоров.

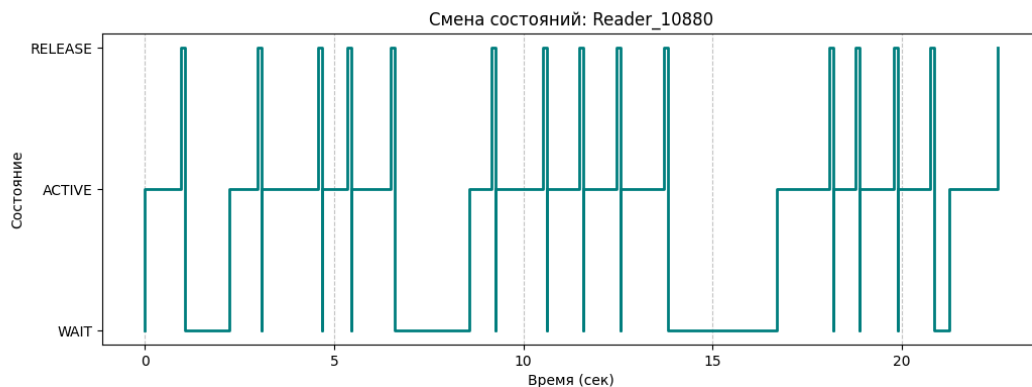


Рис: 23. График пермены состояний читателя pid 12280

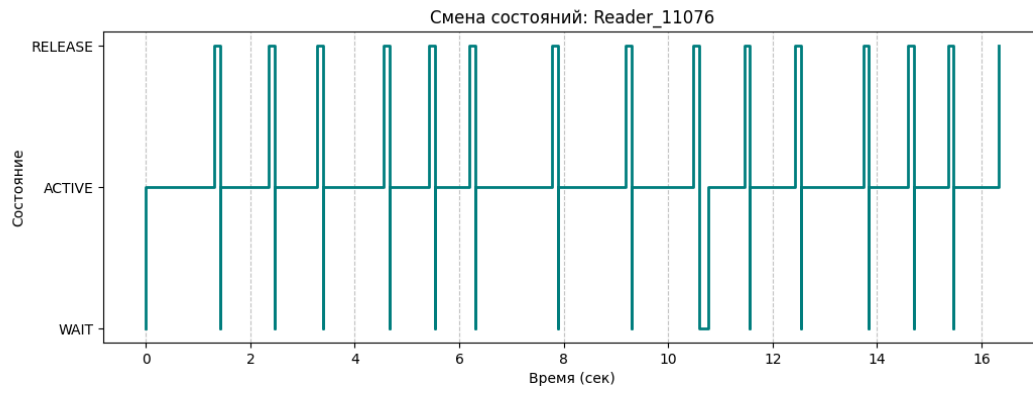


Рис: 24. График пермены состояний читателя рід 13588

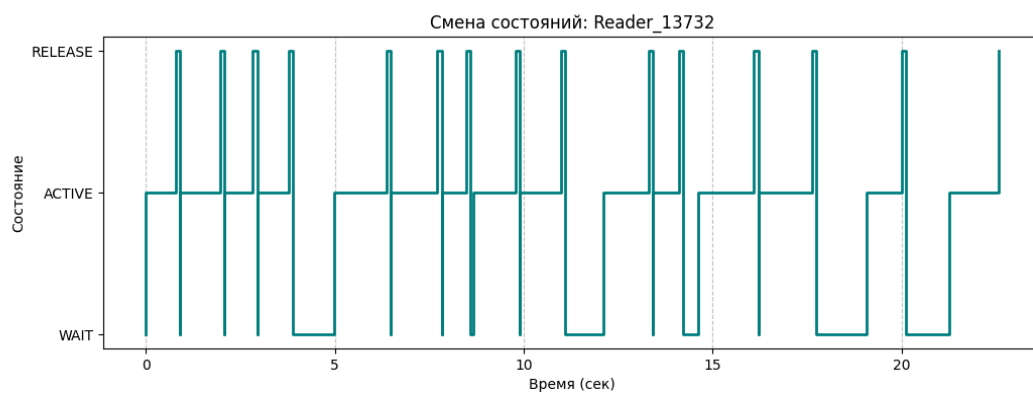


Рис: 25. График пермены состояний читателя рід 15388

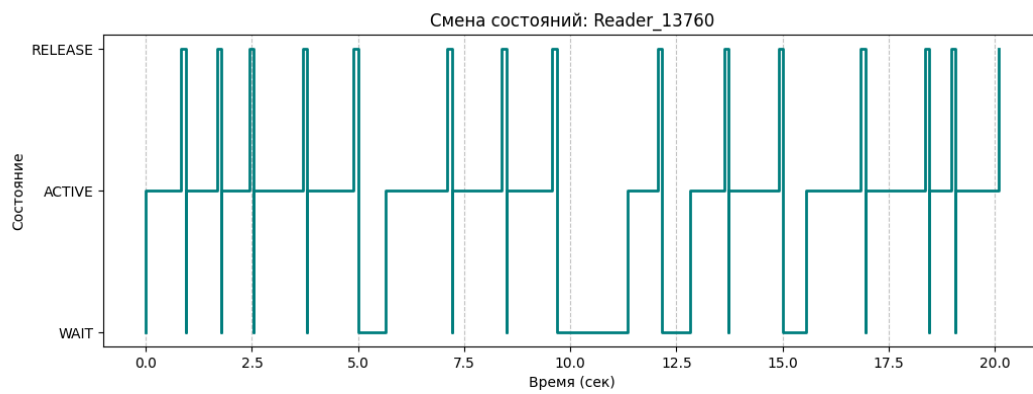


Рис: 26. График пермены состояний читателя рід 16884

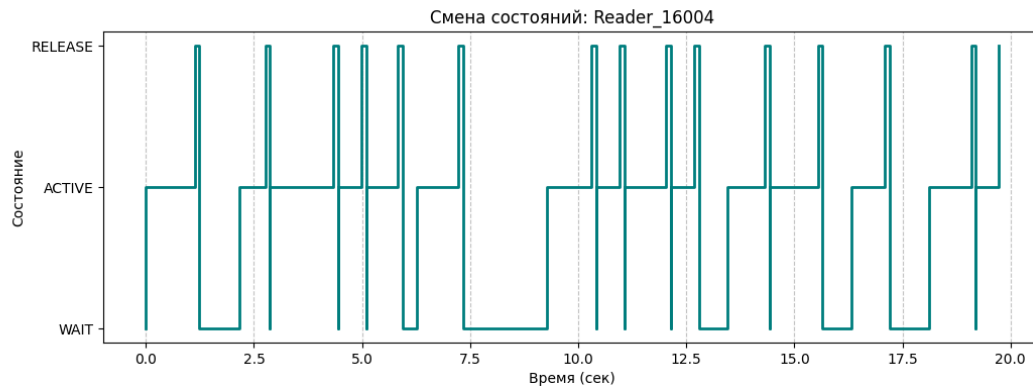


Рис: 27. График пермены состояний читателя рід 2112



Рис: 28. График пермены состояний читателя рід 2716

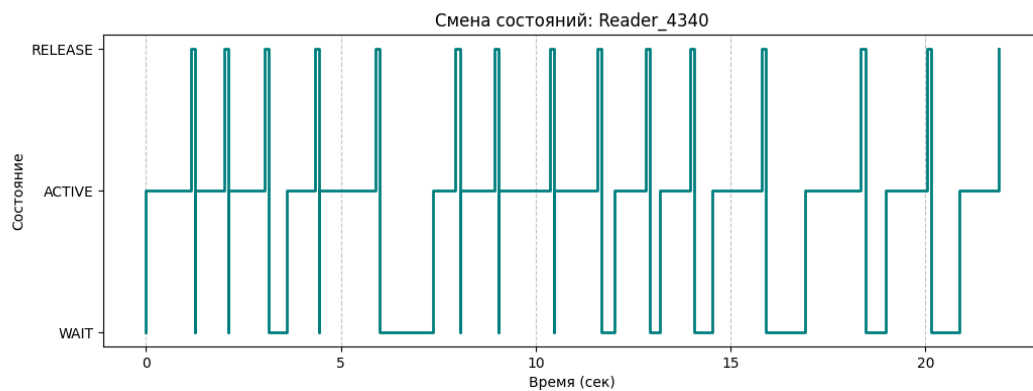


Рис: 29. График пермены состояний читателя рід 3052

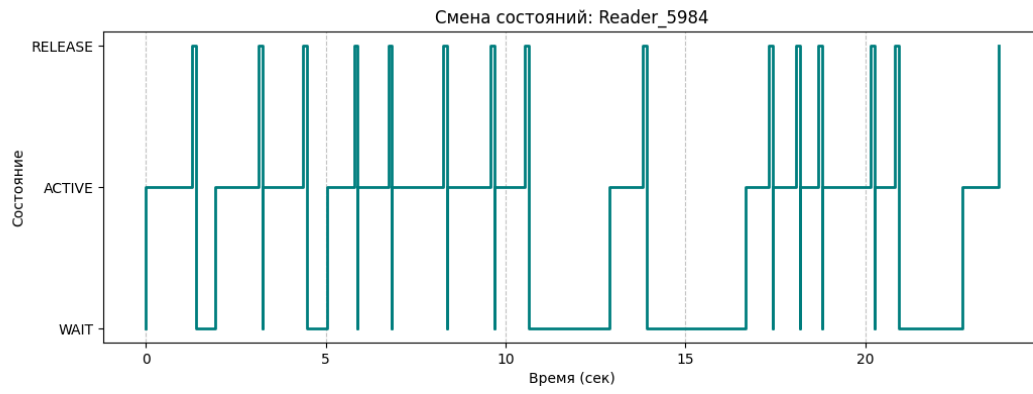


Рис: 30. График пермены состояний читателя rid 3112

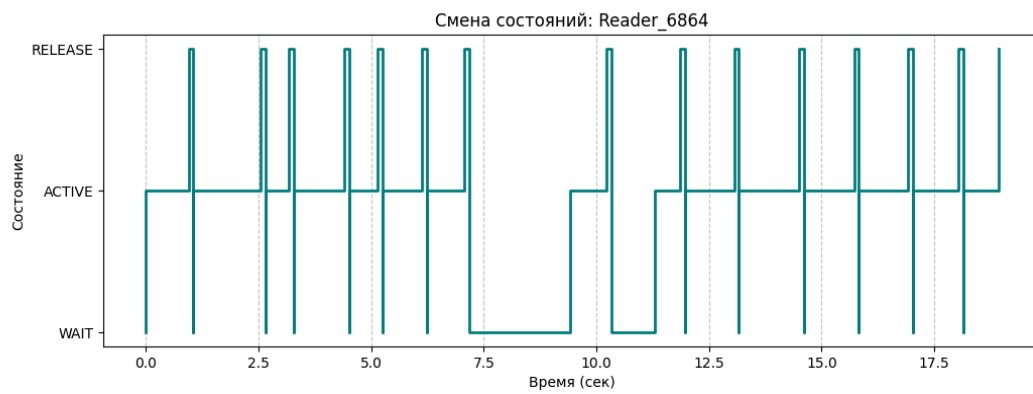


Рис: 31. График пермены состояний читателя rid 4960

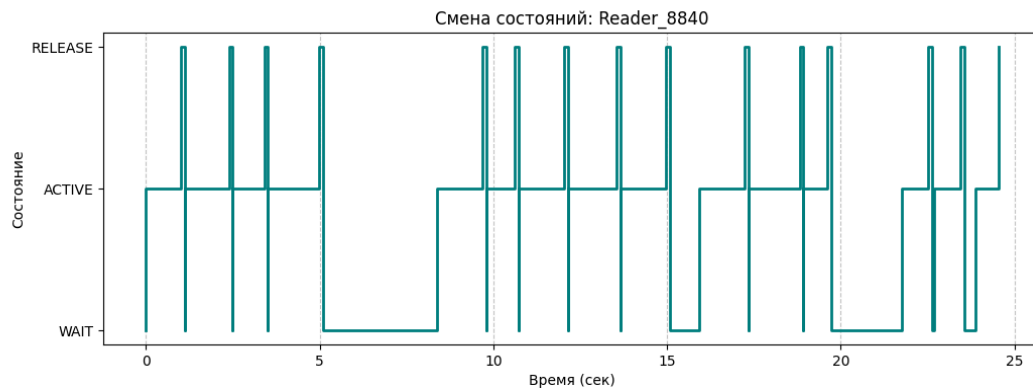


Рис: 32. График пермены состояний читателя rid 7024

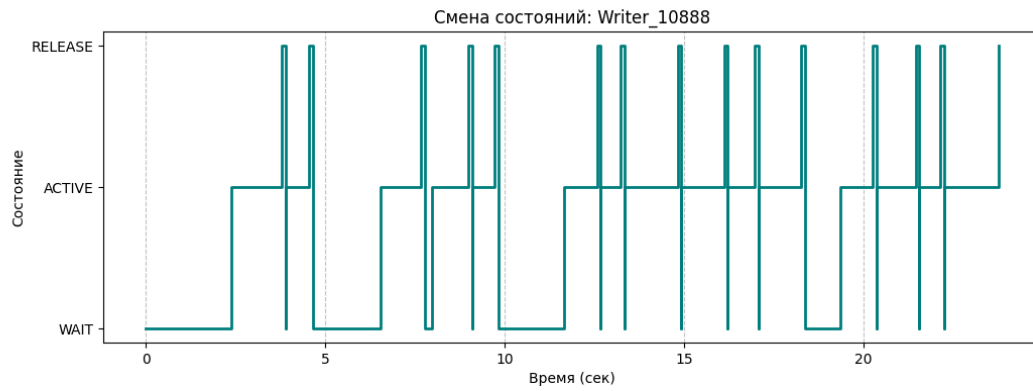


Рис: 33. График пермены состояний писателя pid 10120

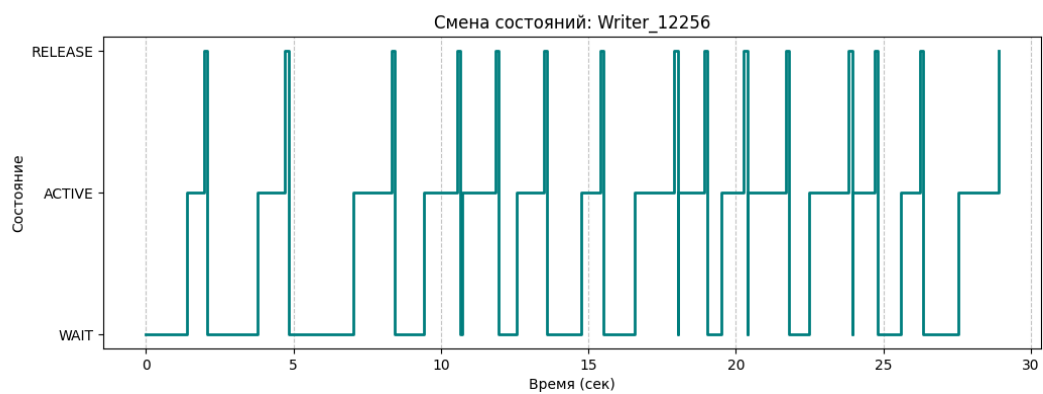


Рис: 34. График пермены состояний писателя pid 13000

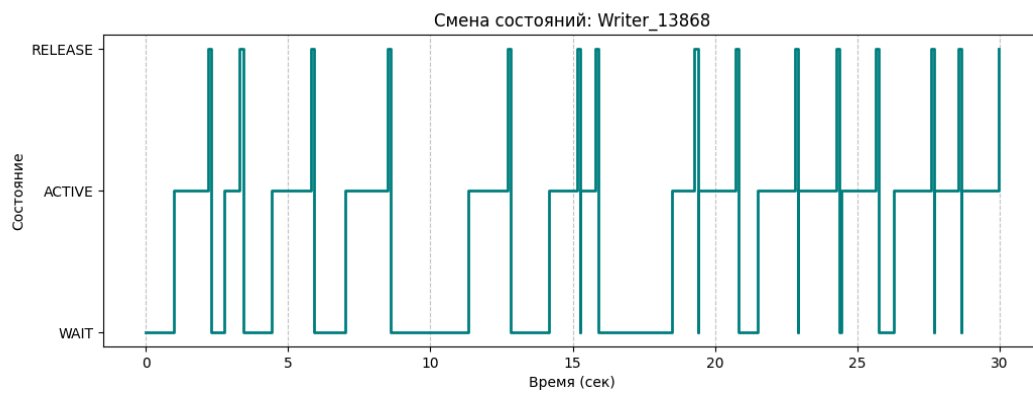


Рис: 35. График пермены состояний писателя pid 13104

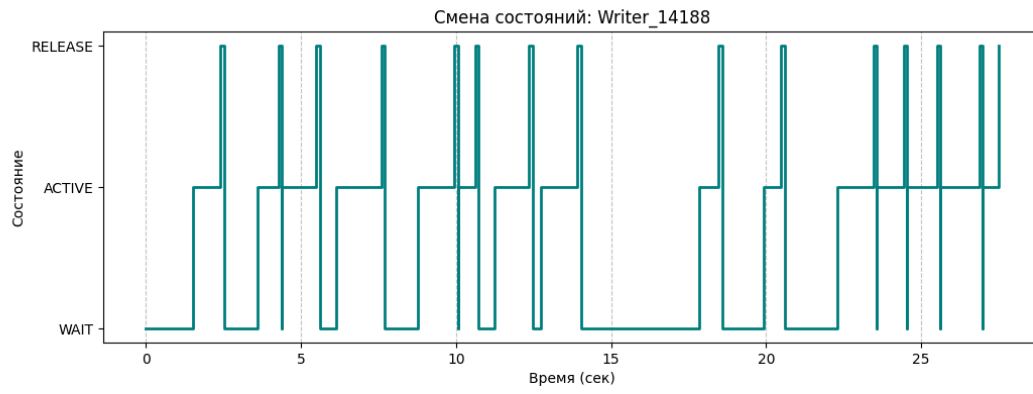


Рис: 36. График пермены состояний писателя pid 4644

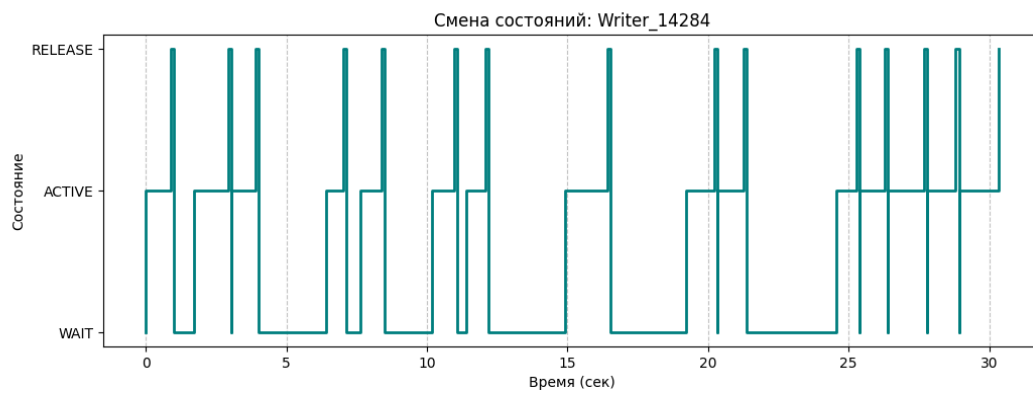


Рис: 37. График пермены состояний писателя pid 5076

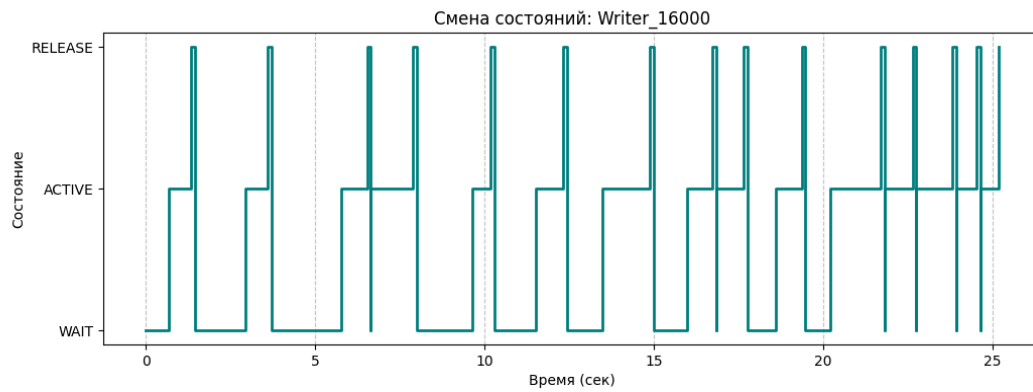


Рис: 38. График пермены состояний писателя pid 5372

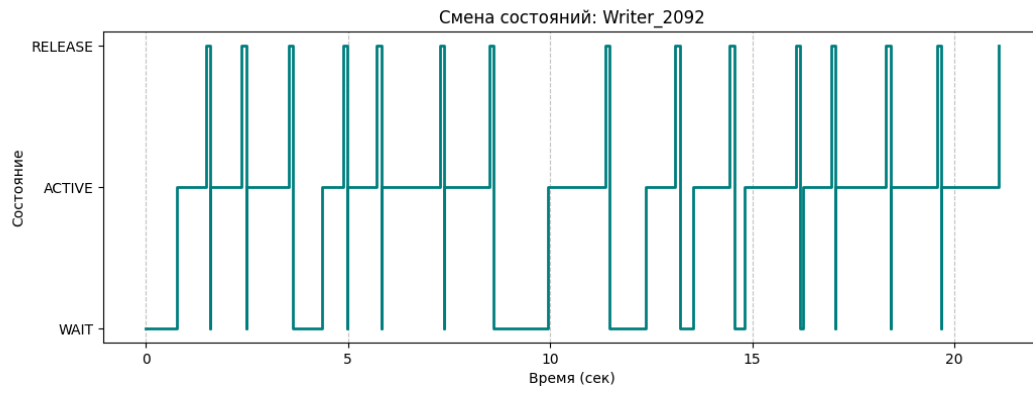


Рис: 39. График пермены состояний писателя pid 6092



Рис: 40. График пермены состояний писателя pid 6788

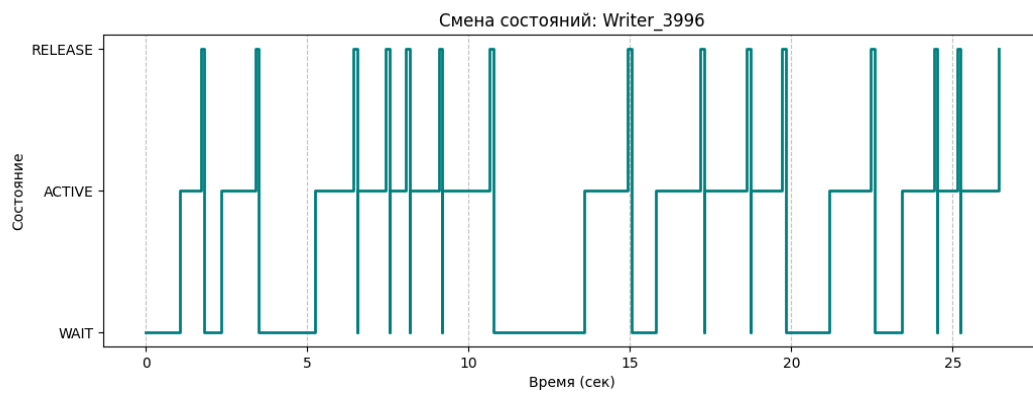


Рис: 41. График пермены состояний писателя pid 8860

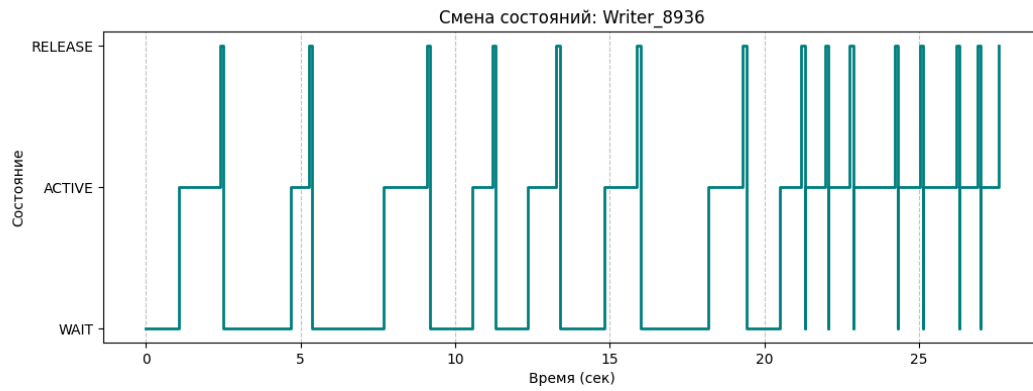


Рис: 42. График пермены состояний писателя pid 16096

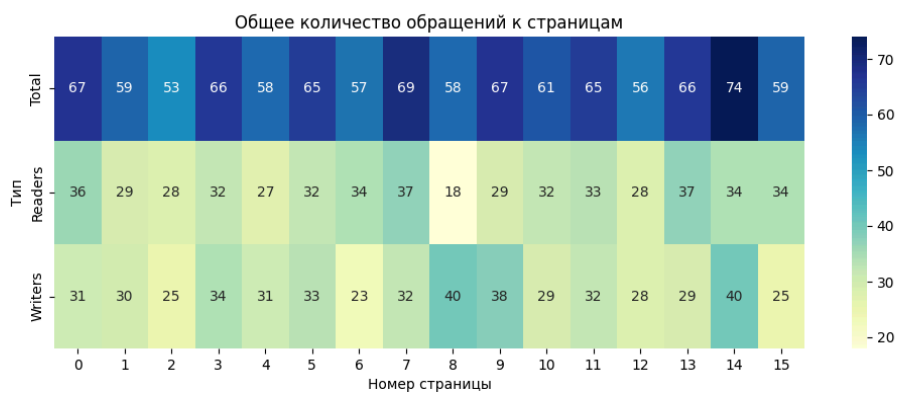


Рис: 43. heatmap обращений к старницам за всё время эксперимента 2

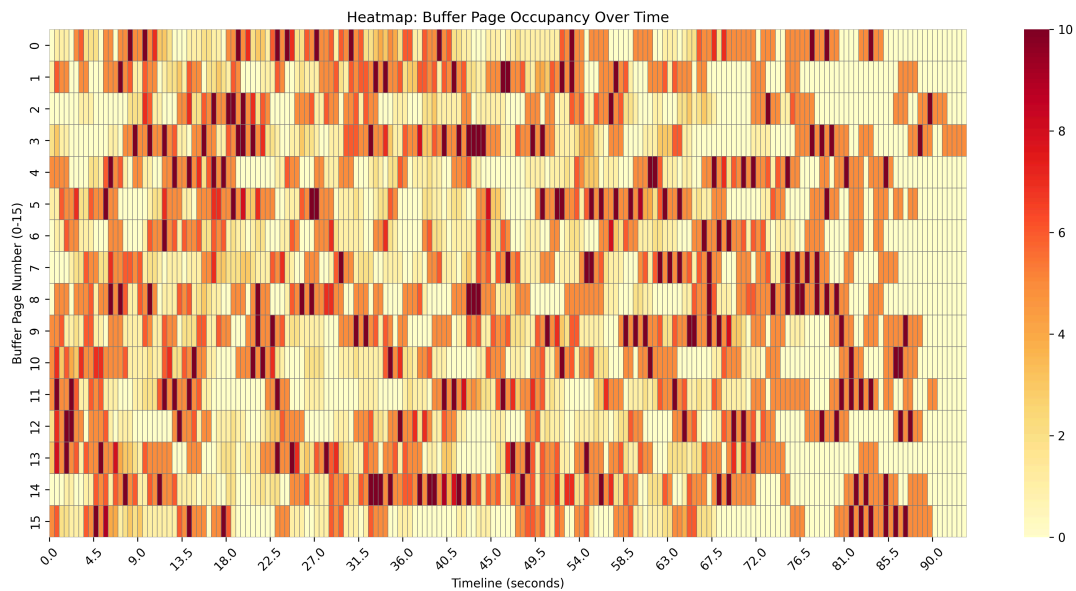


Рис: 44. heatmap обращений к старницам во времени в ходе эксперимента 2

Выводы

В ходе работы проведены два эксперимента с задачей читателей-писателей с использованием проецируемого файла (16 страниц, заблокированы в ОЗУ). В каждом эксперименте одновременно работали 20 процессов (10 читателей, 10 писателей), но с разным числом итераций: первый – 15, второй – 50 итераций на процесс. Длительность операций чтения/записи задавалась случайно в диапазоне 500–1500 мс, синхронизация обеспечивалась семафорами на каждую страницу и мьютексами для счётчика читателей.

Анализ логов показал, что в подавляющем большинстве случаев алгоритм работает корректно: отсутствуют одновременные записи на одной странице, писатели ждут освобождения, читатели могут работать параллельно. Временные метки `timeGetTime` подтверждают соблюдение заданных задержек. Однако в обоих экспериментах (при 15 и при 50 итерациях) в логах обнаружены единичные пересечения активности читателя и писателя на одной странице длительностью около 15 мс. Это значение сопоставимо с разрешающей способностью `timeGetTime` (обычно 1–10 мс) и может быть объяснено погрешностью измерения, а также задержками планировщика ОС. Такие микроскопические наложения не нарушают общей корректности синхронизации и не приводят к состоянию гонки или повреждению данных. Все 20 процессов успешно завершили заданное число итераций без взаимоблокировок.

Таким образом, реализованный алгоритм читателей-писателей с блокировкой страниц в памяти устойчиво работает как при 15, так и при 50 итерациях. Выявленные микроскопические пересечения (≤ 15 мс) лежат в пределах погрешности измерений и не свидетельствуют о системной ошибке синхронизации. Полученные логи и визуализации соответствуют требованиям задания.

Задание 4.1 Реализация решения задачи о читателях-писателях.

Задание

1. **Постановка задачи:** Разработать консольные приложения «Читатель» и «Писатель» для моделирования классической задачи синхронизации при работе с общим ресурсом[cite: 3, 5].
2. **Организация буферной памяти:**
 - Взаимодействие процессов осуществляется через общую буферную память в виде проецируемого файла[cite: 8].
 - Размер одной страницы буфера должен соответствовать размеру физической страницы оперативной памяти[cite: 9].
 - Количество страниц в буфере определяется как сумма цифр номера студенческого билета (без учета первой цифры)[cite: 10].
 - Все страницы буферной памяти должны быть жестко заблокированы в ОЗУ с помощью функции VirtualLock[cite: 11].
3. **Алгоритм работы процессов:**
 - Длительность операций чтения и записи для каждого процесса выбирается случайным образом в интервале от 0,5 до 1,5 секунды[cite: 12, 15].
 - Синхронизация доступа к ресурсу должна быть реализована с помощью объектов Win32 API: семафоров или мьютексов[cite: 16, 17].
4. **Журналирование (логирование):**
 - Каждый экземпляр процесса обязан вести собственный журнальный файл[cite: 18].
 - В журнале фиксируются смены состояний: начало ожидания, выполнение операции (чтение/запись), переход к освобождению[cite: 18].
 - Для каждой записи фиксируется временная метка через TimeGetTime и номер используемой страницы буфера[cite: 18, 19].
5. **Проведение эксперимента:**
 - Необходимо обеспечить одновременную работу группы процессов, общее количество которых не меньше расчетного числа страниц буфера[cite: 20].
 - На основании логов требуется построить графики смены состояний (минимум для 5 читателей и 5 писателей) и тепловую карту (heatmap) занятости страниц[cite: 21, 24, 25].

Описание реализации

Механизм синхронизации

Главным инструментом ограничения доступа к страницам является именованный семафор (префикс LocalPageWriteSem), который создается для каждой страницы отдельно.

- В данной задаче семафор используется в режиме двоичного замка (с максимальным значением 1).
- Для писателя этот семафор является абсолютным блокировщиком: пока писатель удерживает семафор, ни один другой процесс (ни читатель, ни другой писатель) не может получить доступ к этой странице.
- Для читателей этот же семафор служит индикатором присутствия писателя. Читатели захватывают его только коллективно, чтобы «застолбить» страницу и не дать писателю войти, пока идет процесс чтения.

Поскольку специфика задачи допускает одновременное чтение страницы несколькими процессами, возникает проблема: как понять, когда именно нужно заблокировать (или разблокировать) семафор для писателей? Для этого в разделяемой памяти заведен массив `readers_on_page`.

- Однако простое изменение переменной в памяти (`count++`) не является безопасным, так как два читателя могут сделать это одновременно, вызвав состояние гонки.
- Для решения этой проблемы используется именованный мьютекс (префикс LocalPageMutex). Читатель обязан захватить мьютекс перед любым изменением счетчика активных пользователей на странице. Это гарантирует, что только один процесс в конкретный момент времени решает, является ли он «первым вошедшим» (чтобы закрыть семафор от писателей) или «последним вышедшим» (чтобы его открыть).

Если рассматривать процесс синхронизации как поток событий, он выглядит следующим образом:

1. Запрос доступа: Процесс вызывает `WaitForSingleObject`. Если ресурс занят, ядро Windows переводит поток в состояние ожидания, полностью снимая его с выполнения процессором, что исключает «холостой» цикл.
2. Вход в критическую секцию: Как только объект синхронизации освобождается, один из ожидающих процессов (выбранный планировщиком ОС) получает управление. Для читателя это сопровождается захватом мьютекса, проверкой счетчика и, при необходимости, захватом семафора.

3. Безопасная работа: В состоянии активности процесс может быть уверен, что выбранная страница заблокирована согласно его правам (чтение или запись).
4. Освобождение: Процесс вызывает ReleaseMutex или ReleaseSemaphore. Это мгновенно переводит объект синхронизации в сигнальное состояние, пробуждая другие ожидающие процессы.

Читатели

Алгоритм «Читателя» более сложен, так как он поддерживает режим коллективного доступа к данным, блокируя только писателей. После стандартной процедуры инициализации и отображения файла в память, читатель вступает в цикл обработки данных.

В начале каждой итерации процесс выбирает случайную страницу и фиксирует в логе состояние ожидания (код 0). Для координации действий читатель использует два объекта синхронизации: мьютекс для защиты общего счетчика и семафор для блокировки записи. Сначала процесс захватывает именованный мьютекс, чтобы безопасно инкрементировать значение `readers_on_page` в разделяемой памяти. Если после инкремента выясняется, что данный читатель является первым на странице, он вызывает `WaitForSingleObject` для семафора записи, тем самым закрывая доступ писателям на все время, пока страница будет занята хотя бы одним читателем.

После освобождения мьютекса процесс переходит в состояние активности (код 1). Он считывает содержимое страницы, выводит его на экран и имитирует выполнение операции в течение заданного случайного интервала. По завершении чтения фиксируется состояние перехода к освобождению (код 2). Процесс снова захватывает мьютекс страницы и уменьшает счетчик активных читателей. Если текущий читатель оказывается последним (счетчик становится равен нулю), он вызывает `ReleaseSemaphore`, сигнализируя о том, что страница полностью свободна и доступна для писателей. В завершение итерации закрываются хендлы объектов синхронизации, и процесс готовится к новому циклу чтения.

Писатели

Процесс «Писатель» реализует стратегию строгого эксклюзивного доступа к ресурсу. Работа приложения начинается с инициализации системных параметров, где через функцию `GetSystemInfo` определяется размер физической страницы памяти. Далее выполняется проецирование файла `buffer_storage.bin` в адресное пространство процесса с обязательной фиксацией страниц в ОЗУ через `VirtualLock`.

Основной цикл работы писателя состоит из пятнадцати итераций. На каждом этапе процесс случайным образом выбирает целевую страницу буфера в диапазоне от 0 до 15. Перед началом модификации данных писатель переходит в состояние ожидания (код 0), запрашивая доступ к именованному семафору, уникальному для выбранной страницы. Использование функции `WaitForSingleObject` гарантирует, что запись

начнется только тогда, когда на странице отсутствуют любые другие процессы — как читатели, так и другие писатели.

При получении доступа процесс переходит в состояние активности (код 1). В журнальный файл записывается метка времени `timeGetTime` и номер рабочей страницы. Писатель формирует информационную строку, содержащую его идентификатор и текущее время, после чего записывает её в буфер. Для имитации реальной нагрузки выполняется задержка от 0,5 до 1,5 секунд. Завершив операцию, процесс регистрирует состояние освобождения (код 2), отпускает семафор функцией `ReleaseSemaphore` и делает короткую паузу перед следующей итерацией, позволяя другим процессам перехватить ресурс.

Журналирование

Журналирование в данной работе организовано как децентрализованная система сбора данных, где каждый активный процесс самостоятельно фиксирует свою активность в отдельном файле формата CSV. Это позволяет детально восстановить хронологию событий при последующем анализе и построении графиков.

Для исключения конфликтов при записи каждый процесс создает уникальный файл, в названии которого фигурирует роль приложения («Reader» или «Writer») и его системный идентификатор (PID). Например, файл может называться `Writer_1234.csv`. Данные записываются в текстовом формате, где значения разделены запятыми, что упрощает их последующий импорт в Excel или Python для визуализации.

Каждая строка журнала соответствует одному событию и содержит три обязательных поля:

1. **Временная метка (Timestamp):** Получается с помощью функции Win32 API `timeGetTime`, которая возвращает количество миллисекунд, прошедших с момента старта операционной системы. Это обеспечивает высокую точность синхронизации событий между разными процессами.
2. **Код состояния (State):** Числовой идентификатор, описывающий текущий этап работы процесса:
 - 0 — **Ожидание:** процесс запросил доступ к странице и заблокирован планировщиком.
 - 1 — **Активность:** процесс успешно захватил объекты синхронизации и выполняет чтение или запись данных.
 - 2 — **Освобождение:** процесс завершил работу с данными и готовится отпустить семафор или мьютекс.
3. **Номер страницы:** Индекс страницы (от 0 до 15), с которой в данный момент взаимодействует процесс.

Процесс записи реализован через вспомогательную функцию LogCsv, которая принимает поток файла, код состояния и номер страницы. Важной особенностью реализации является момент фиксации событий:

1. Событие **«Ожидание»** регистрируется непосредственно **перед** вызовом блокирующей функции WaitForSingleObject.
2. Событие **«Активность»** записывается сразу **после** выхода из функции ожидания, что позволяет точно замерить время простоя процесса в очереди.
3. Событие **«Освобождение»** фиксируется **перед** реальным отпусканьем объектов синхронизации (ReleaseSemaphore / ReleaseMutex), чтобы гарантировать, что в логах период активности не перекрывается с моментом, когда страницу уже занял другой поток.

Собранные данные служат фундаментом для аналитической части отчета. На их основе строятся сводные графики состояний, позволяющие визуальнo оценить переходы процессов между ожиданием и работой и тепловые карты (Heatmaps) занятости страниц, которые наглядно демонстрируют распределение нагрузки на буферную память и эффективность механизмов синхронизации в предотвращении коллизий.

Результаты экспериментов

В ходе работы было проведено два независимых эксперимента, различающихся интенсивностью нагрузки на систему. Для каждого эксперимента был сформирован полный пакет отчетной документации, включающий 20 графиков смены состояний (для каждого из 10 читателей и 10 писателей) и 2 тепловые карты занятости страниц.

Эксперимент №1: Базовая проверка (15 итераций)

Первый этап тестирования был направлен на подтверждение корректности логики работы алгоритмов в штатном режиме. При 15 итерациях на каждый процесс общая продолжительность работы составила около 30–40 секунд.

Цель: Проверка отсутствия ошибок при создании и открытии объектов синхронизации, а также контроль корректности записи логов.

Наблюдения: На графиках и хитмапах данного этапа фиксируется стабильная работа без длительных задержек. Ресурсы освобождаются достаточно быстро, чтобы новые процессы могли занимать страницы без формирования глубоких очередей.



Рис: 45. График пермены состояний читателя рід 12280



Рис: 46. График пермены состояний читателя рід 13588



Рис: 47. График пермены состояний читателя рід 15388



Рис: 48. График пермены состояний читателя рід 16884

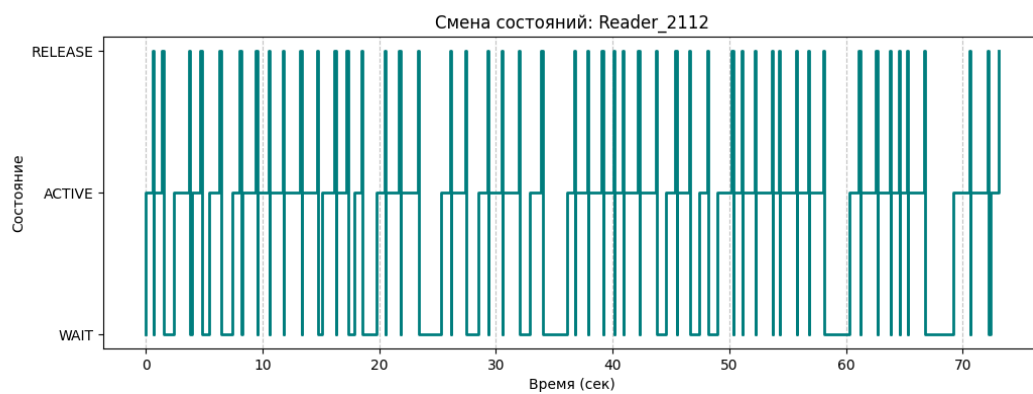


Рис: 49. График пермены состояний читателя рід 2112



Рис: 50. График пермены состояний читателя рід 2716



Рис: 51. График пермены состояний читателя pid 3052



Рис: 52. График пермены состояний читателя pid 3112



Рис: 53. График пермены состояний читателя pid 4960



Рис: 54. График пермены состояний читателя pid 7024



Рис: 55. График пермены состояний писателя pid 10120



Рис: 56. График пермены состояний писателя pid 13000



Рис: 57. График пермены состояний писателя pid 13104

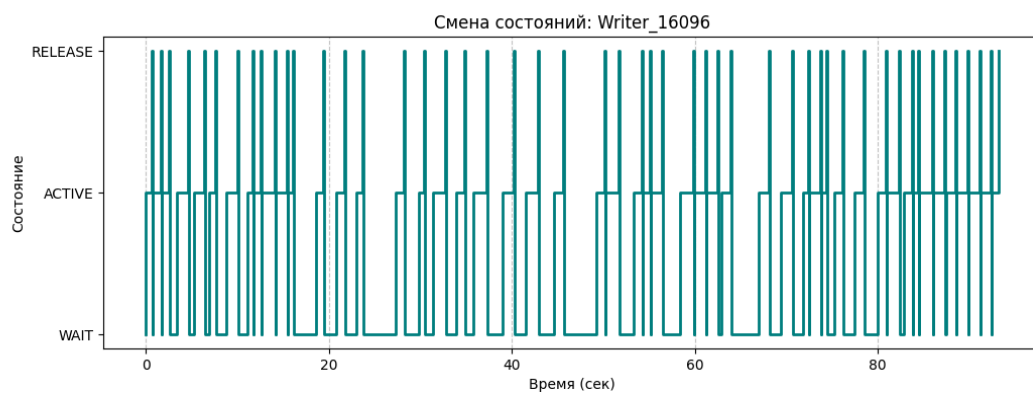


Рис: 58. График пермены состояний писателя pid 16096



Рис: 59. График пермены состояний писателя pid 4644



Рис: 60. График пермены состояний писателя pid 5076



Рис: 61. График пермены состояний писателя pid 5372

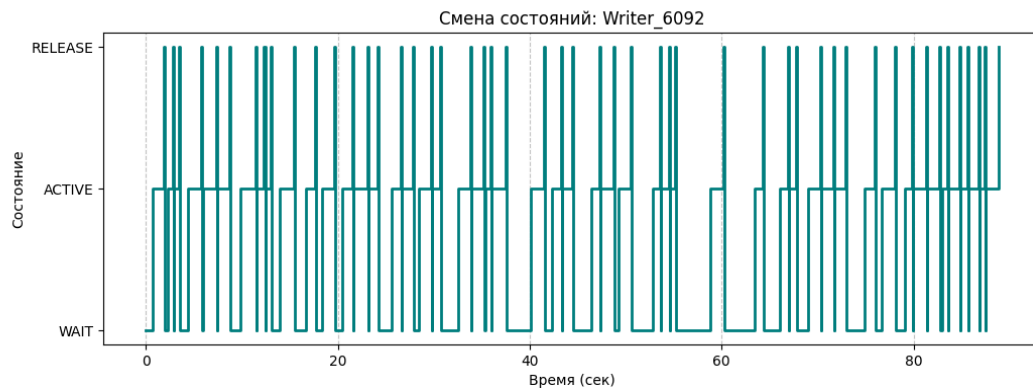


Рис: 62. График пермены состояний писателя pid 6092



Рис: 63. График пермены состояний писателя рід 6788



Рис: 64. График пермены состояний писателя рід 8860

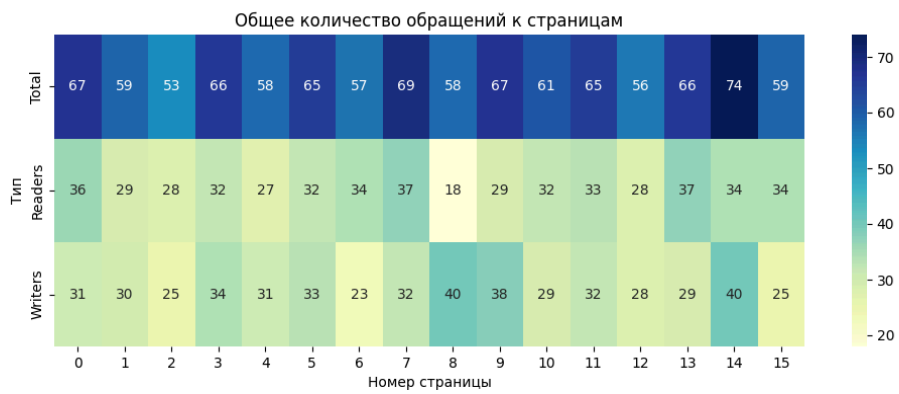


Рис: 65. heatmap обращений к страницам за всё время эксперимента 1

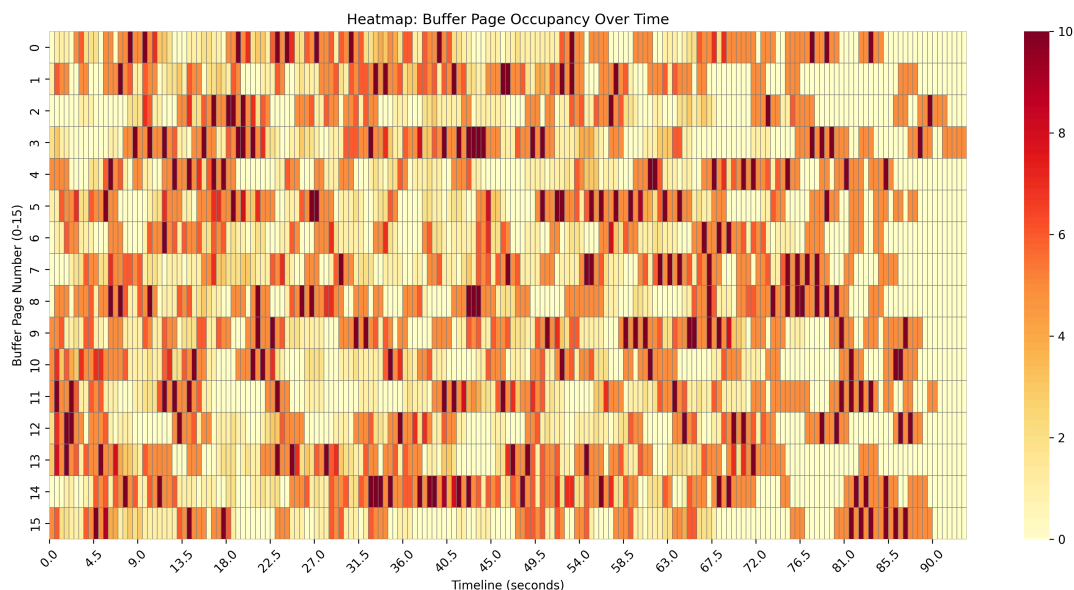


Рис: 66. heatmap обращений к страницам во времени в ходе эксперимента 1

Эксперимент №2: Расширенное тестирование (50 итераций)

Второй этап проводился при тех же 20 активных процессах, но с увеличением числа итераций до 50 для каждого. Это позволило перевести систему в режим длительного функционирования.

Цель: Изучение поведения механизмов синхронизации при накоплении запросов и проверка устойчивости проецируемого файла к длительным блокировкам.

Наблюдения: Увеличение объема данных позволило более наглядно визуализировать работу планировщика ОС. На графиках состояний отчетливо видны участвовавшие периоды ожидания (код 0), что характерно для условий высокой конкуренции за популярные страницы буфера. Тепловые карты за этот период демонстрируют более плотное заполнение временной шкалы, подтверждая надежность работы исключаящих семафоров.

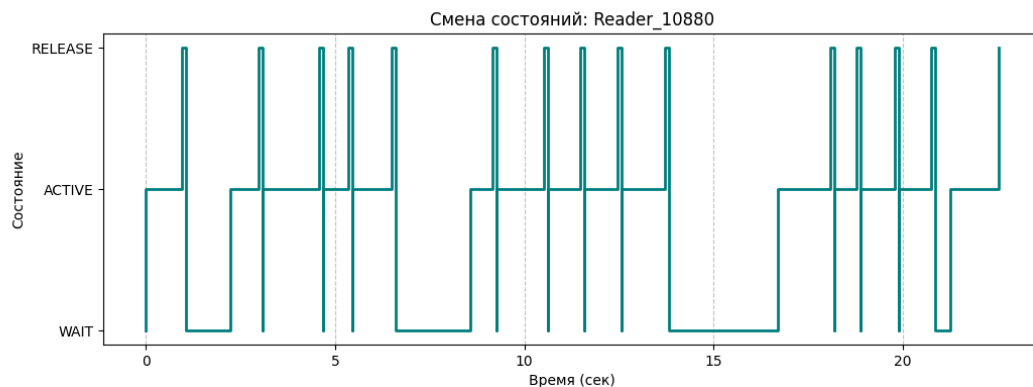


Рис: 67. График пермены состояний читателя pid 12280

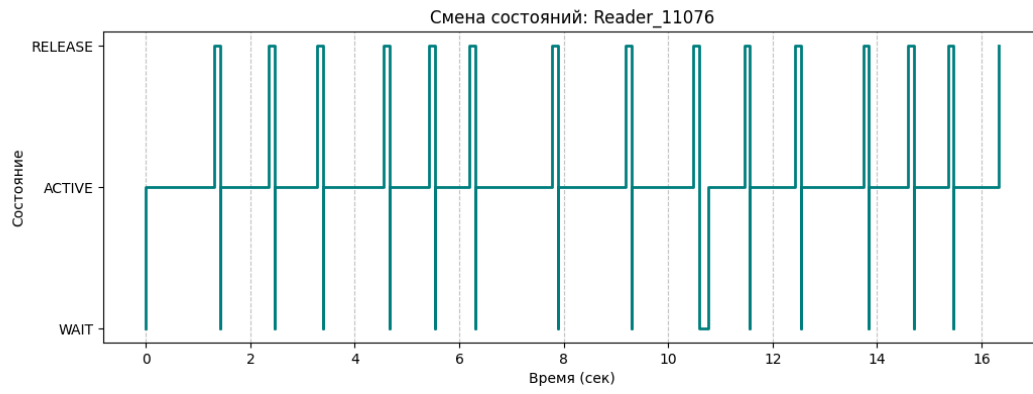


Рис: 68. График пермены состояний читателя рід 13588

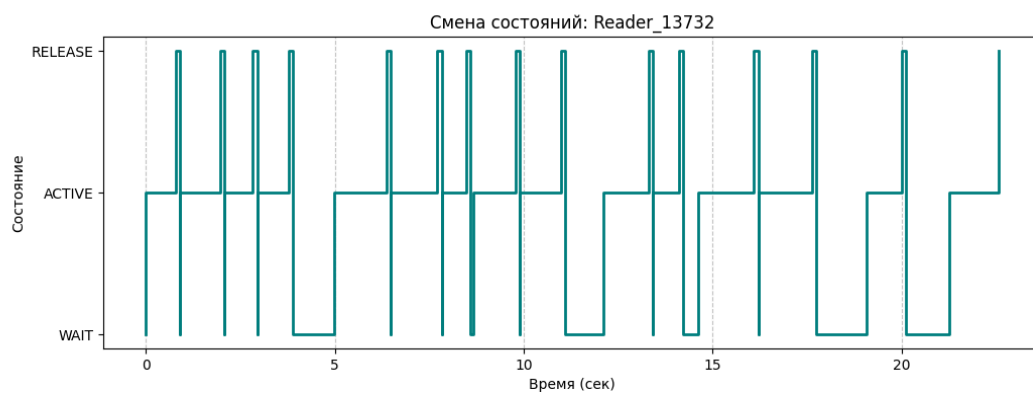


Рис: 69. График пермены состояний читателя рід 15388

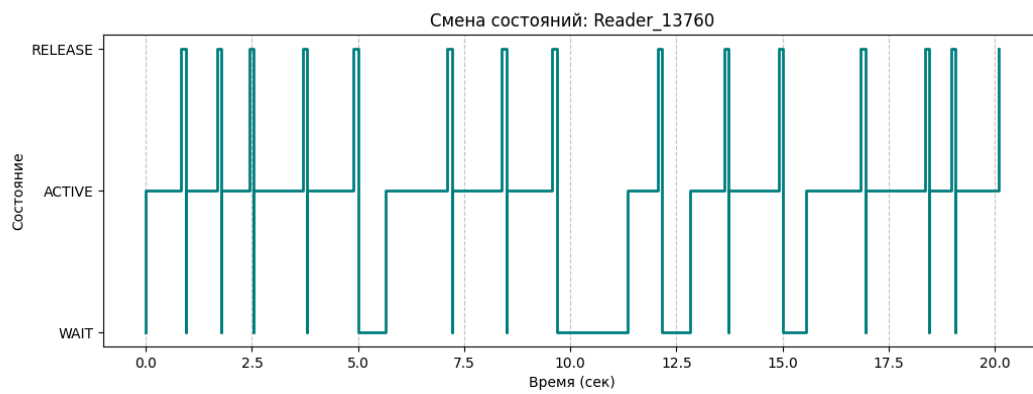


Рис: 70. График пермены состояний читателя рід 16884

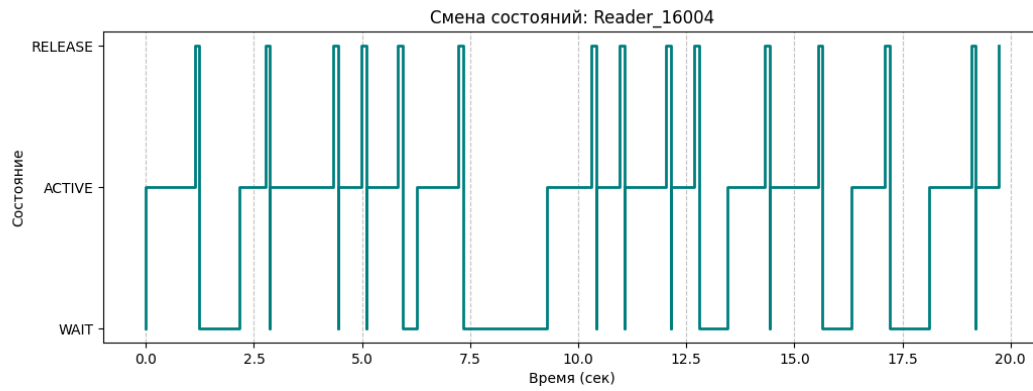


Рис: 71. График пермены состояний читателя рід 2112



Рис: 72. График пермены состояний читателя рід 2716

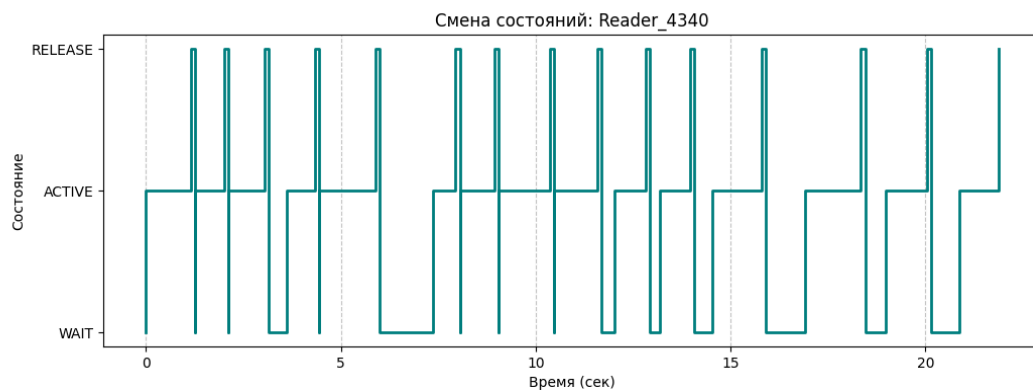


Рис: 73. График пермены состояний читателя рід 3052

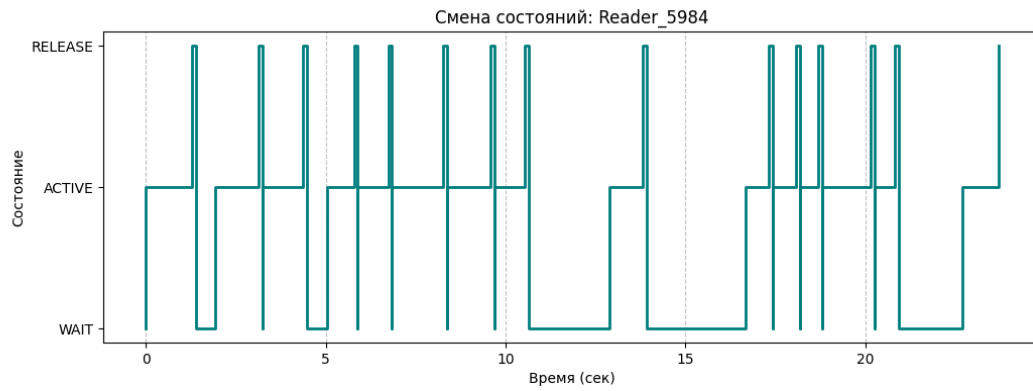


Рис: 74. График пермены состояний читателя pid 3112

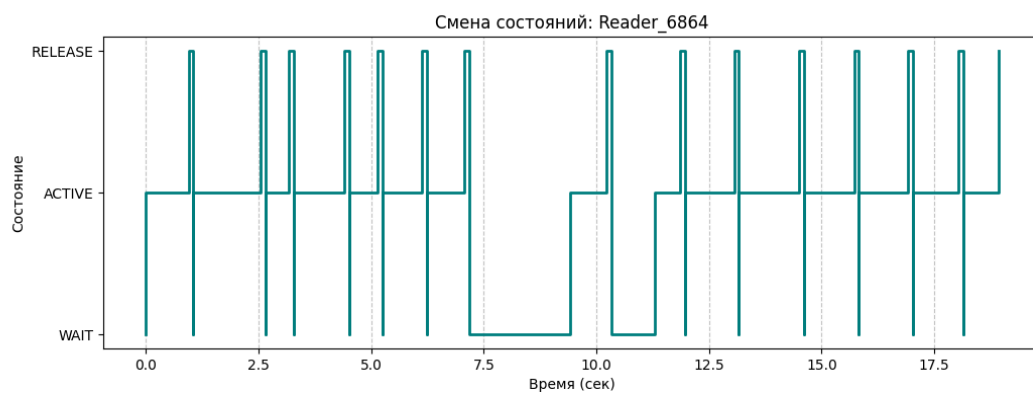


Рис: 75. График пермены состояний читателя pid 4960

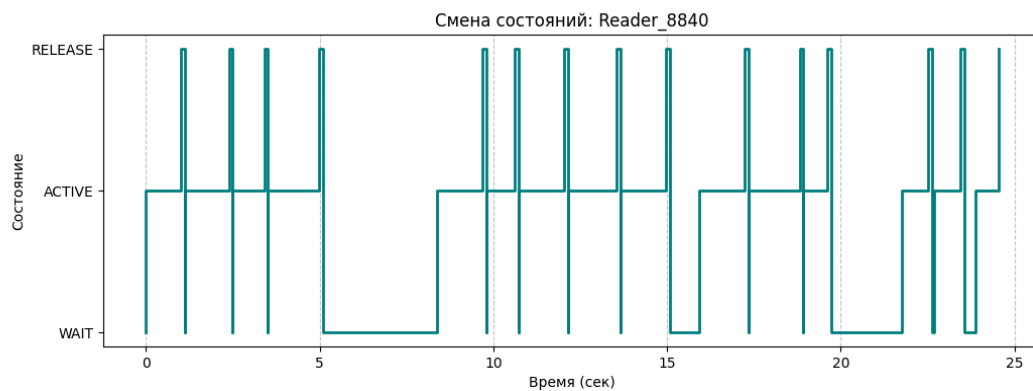


Рис: 76. График пермены состояний читателя pid 7024

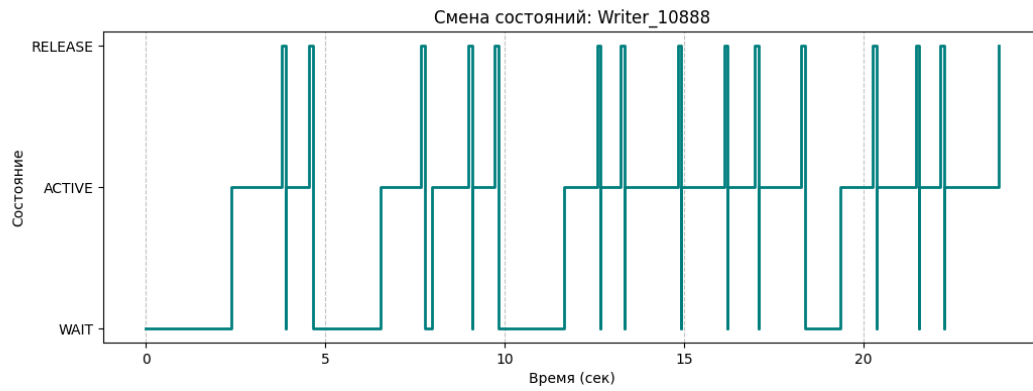


Рис: 77. График пермены состояний писателя pid 10120

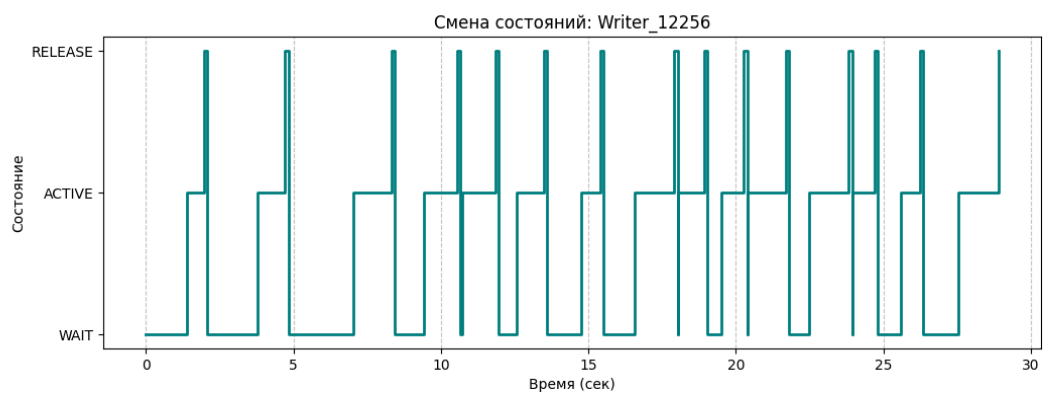


Рис: 78. График пермены состояний писателя pid 13000

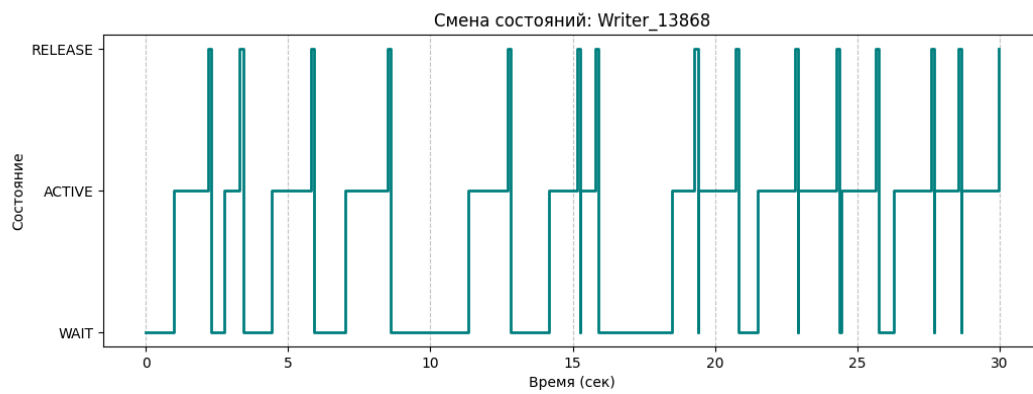


Рис: 79. График пермены состояний писателя pid 13104

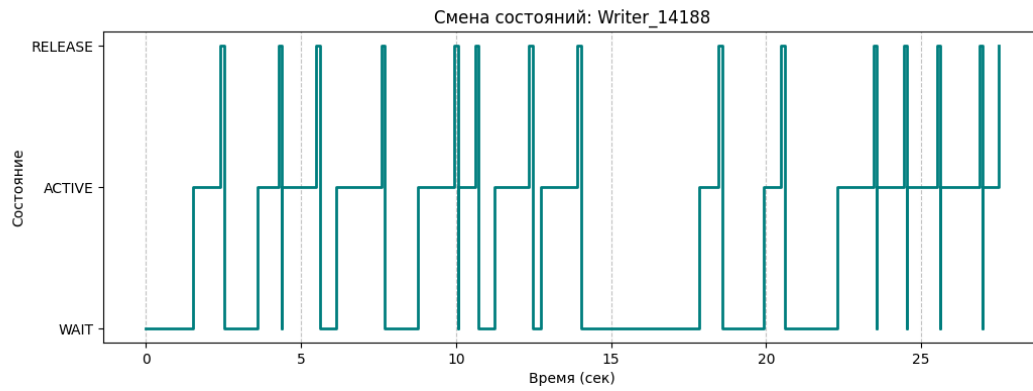


Рис: 80. График пермены состояний писателя pid 4644

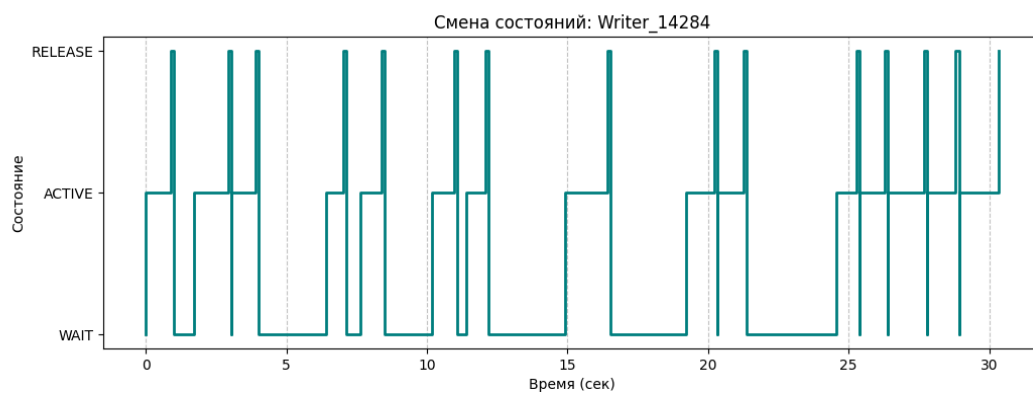


Рис: 81. График пермены состояний писателя pid 5076

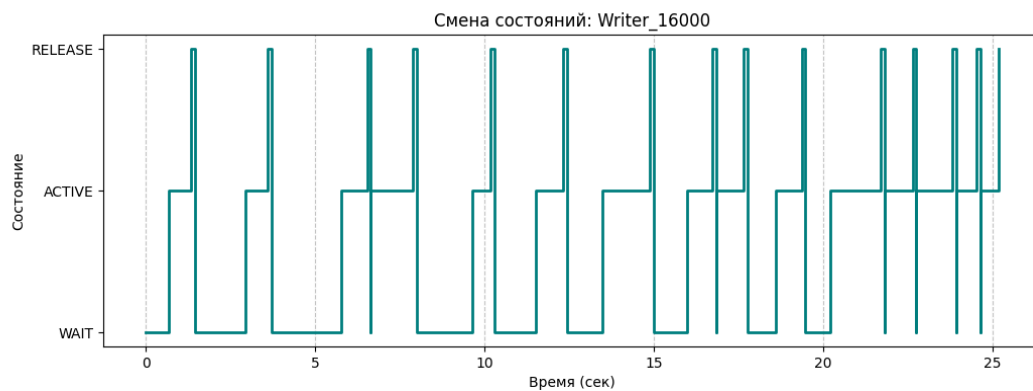


Рис: 82. График пермены состояний писателя pid 5372

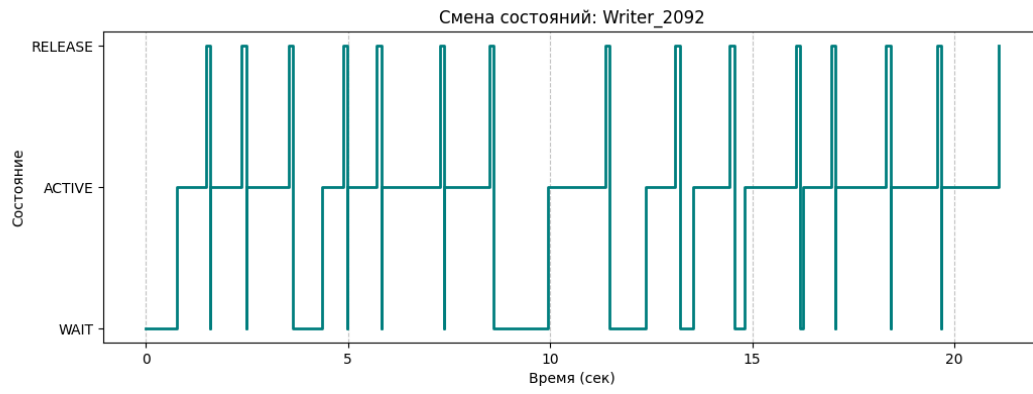


Рис: 83. График пермены состояний писателя pid 6092



Рис: 84. График пермены состояний писателя pid 6788

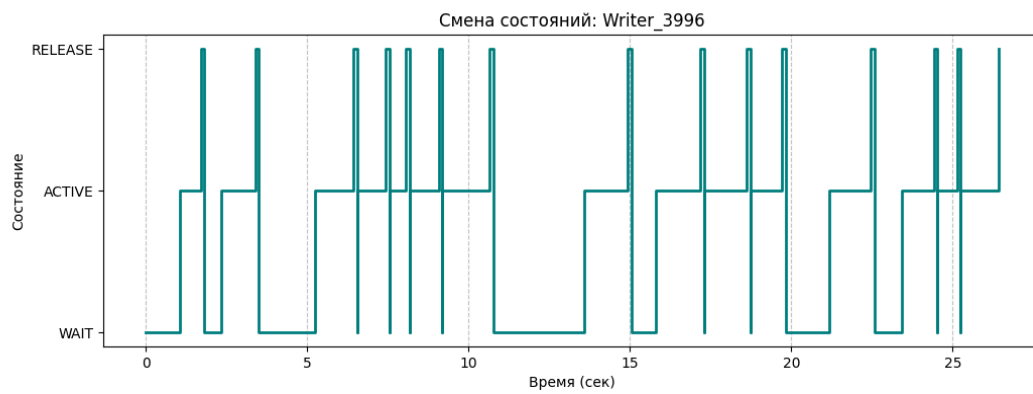


Рис: 85. График пермены состояний писателя pid 8860

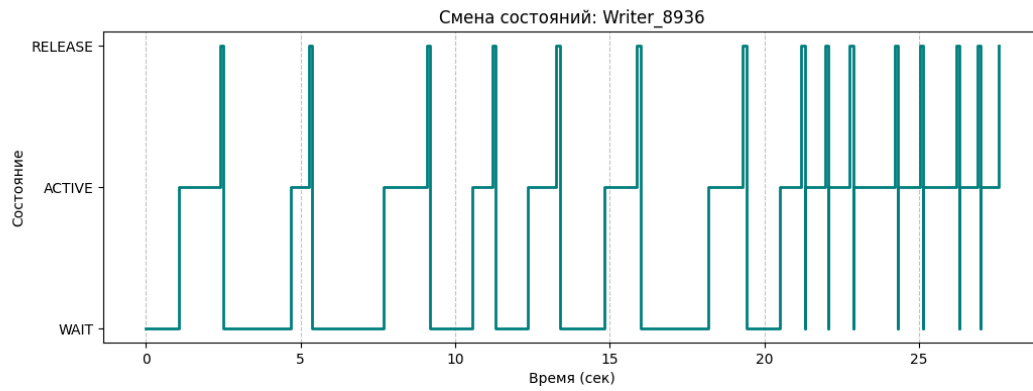


Рис: 86. График пермены состояний писателя pid 16096

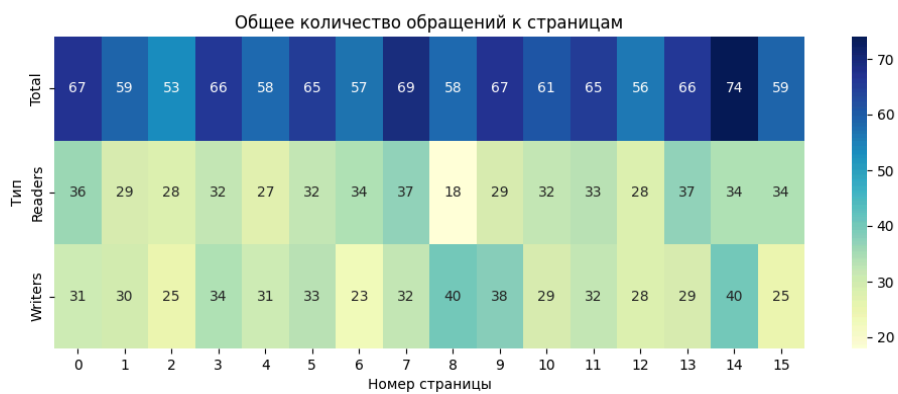


Рис: 87. heatmap обращений к старницам за всё время эксперимента 2

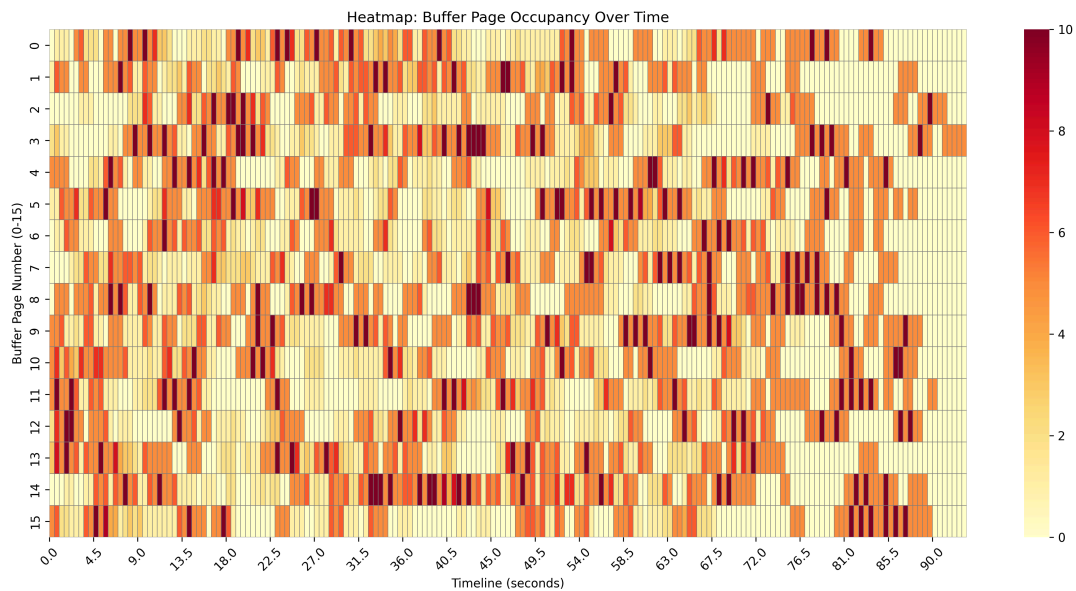


Рис: 88. heatmap обращений к старницам во времени в ходе эксперимента 2

Выводы

В ходе работы проведены два эксперимента с задачей читателей-писателей с использованием проецируемого файла (16 страниц, заблокированы в ОЗУ). В каждом эксперименте одновременно работали 20 процессов (10 читателей, 10 писателей), но с разным числом итераций: первый – 15, второй – 50 итераций на процесс. Длительность операций чтения/записи задавалась случайно в диапазоне 500–1500 мс, синхронизация обеспечивалась семафорами на каждую страницу и мьютексами для счётчика читателей.

Анализ логов показал, что в подавляющем большинстве случаев алгоритм работает корректно: отсутствуют одновременные записи на одной странице, писатели ждут освобождения, читатели могут работать параллельно. Временные метки `timeGetTime` подтверждают соблюдение заданных задержек. Однако в обоих экспериментах (при 15 и при 50 итерациях) в логах обнаружены единичные пересечения активности читателя и писателя на одной странице длительностью около 15 мс. Это значение сопоставимо с разрешающей способностью `timeGetTime` (обычно 1–10 мс) и может быть объяснено погрешностью измерения, а также задержками планировщика ОС. Такие микроскопические наложения не нарушают общей корректности синхронизации и не приводят к состоянию гонки или повреждению данных. Все 20 процессов успешно завершили заданное число итераций без взаимоблокировок.

Таким образом, реализованный алгоритм читателей-писателей с блокировкой страниц в памяти устойчиво работает как при 15, так и при 50 итерациях. Выявленные микроскопические пересечения (≤ 15 мс) лежат в пределах погрешности измерений и не свидетельствуют о системной ошибке синхронизации. Полученные логи и визуализации соответствуют требованиям задания.

Приложение

Исходный код можно найти в репозитории по адресу <https://github.com/AlekseyPozhidaev/OS-LW4>